

Experiment 2

1. Aim

Use var, let, const, and demonstrate basic features like template literals and destructuring; build a billing calculator with user input.

2. Software/Tools Required

- Visual Studio Code (VS Code) or Notepad++
- Google Chrome / Mozilla Firefox / Microsoft Edge
- HTML
- JavaScript

3. Code

```
<!DOCTYPE html>    // Shravasti Shende
<html lang="en">    // 24070521194
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>My Billing Calculator</title>
  <style>
    body {
      background-color: #0004f8; /* Soft blue/teal background */
      font-family: 'Times New Roman', Times, serif;
      margin: 40px;
      color: #eedf1;
    }
    h1 {
      font-size: 2.5rem;
      margin-bottom: 20px;
    }
    h2 {
      font-size: 1.8rem;
      margin-top: 30px;
      margin-bottom: 20px;
    }
  }
```

```
.receipt-line {
  font-size: 1.3rem;
  margin: 15px 0;
}
.bold {
  font-weight: bold;
}
</style>
</head>
<body>
  <div id="app"></div>

  <script>
    // 1. Customize your Title here using 'var'
    var calculatorTitle = "Shravasti's Billing Desk";

    // 2. Collect inputs (Customer Name prompt has been removed!)
    const productName = prompt("Enter Product Name:") || "Item";
    const quantityInput = prompt("Enter Quantity:") || "1";
    const priceInput = prompt("Enter Price per Item:") || "0";

    // Convert types safely
    const quantity = parseInt(quantityInput, 10) || 0;
    const pricePerItem = parseFloat(priceInput) || 0;

    // 3. Destructuring (Cleanly extracting only what we need)
    const invoiceData = {
      product: productName,
      qty: quantity,
      rate: pricePerItem
    };
  </script>
</body>
</html>
```

```

    };
    const { product, qty, rate } = invoiceData;

    // 4. Math calculations
    const rawTotal = qty * rate;
    const gstRate = 18;
    const gstAmount = rawTotal * (gstRate / 100);
    const finalBill = rawTotal + gstAmount;

    // 5. Render directly to the page using Template Literals (Customer line removed)
    const appContainer = document.getElementById('app');
    appContainer.innerHTML = `
        <h1>${calculatorTitle}</h1>
        <h2>Billing Receipt</h2>

        <div class="receipt-line">Product: ${product}</div>
        <div class="receipt-line">Quantity: ${qty}</div>
        <div class="receipt-line">Price per item: ₹${rate}</div>
        <div class="receipt-line">Total: ₹${rawTotal}</div>
        <div class="receipt-line">GST (${gstRate}%): ₹${gstAmount.toFixed(2)}</div>
        <div class="receipt-line bold">Final Bill: ₹${finalBill.toFixed(2)}</div>
    `;
</script>
</body>
</html>

```

4.Theory

JavaScript provides different ways to declare variables:

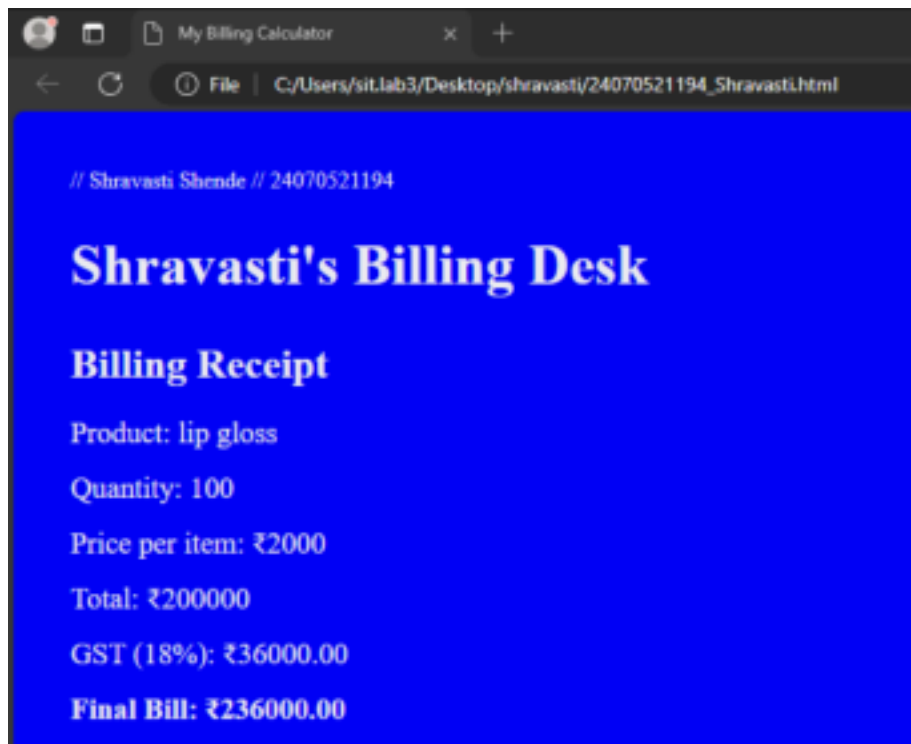
- **var** is function-scoped and can be redeclared and updated.
- **let** is block-scoped and can be updated but not redeclared within the same block.
- **const** is block-scoped and cannot be reassigned after initialization.

Template literals use backticks (```) to create strings and allow embedding variables using `${}`. **Destructuring** is a

JavaScript feature that extracts values from arrays or objects into separate variables.

In this experiment, a billing calculator accepts user input such as item name, quantity, and price. It calculates the total bill and displays the result using template literals while demonstrating the use of `var`, `let`, `const`, and destructuring.

4. Output



5. Result/Conclusion

The experiment was executed successfully. The concepts of var, let, const, template literals, and destructuring were demonstrated, and the billing calculator correctly accepted user input and displayed the total bill.